Bartłomiej Prus

Mateusz Szwagierczak

Dokumentacja

Lokalizacja punktu w przestrzeni dwuwymiarowej – metoda doskonalenia triangulacji – algorytm Kirkpatrick'a

1. **Wymagania techniczne**
   Do poprawnego działania programu konieczne jest posiadanie bibliotek Python'owych:
   a. **numpy** (wykorzystywana do tworzenia tablic i wykonywaniaoperacji na nich)
   b. **matplotlib** (wykorzystywana do wizualizacji)
   c. **scipy** (wykorzystywany do triangulacji Delaunay'a)
   d. **mapbox_earcut** (wykorzystywana do triangulacji Earcut)
   e. **shapely** (wykorzystywana do operacji na obiektach 2d takich jak punkty, odcinki, wielokąty)
   f. **collections** (wykorzystana do struktury defaultdict)

   Dodatkowo do wizualizacji uzyskanych wyników konieczne jest pobranie narzędzia graficznego:
   g. **bitalg** (dostępne na https://github.com/aghbit/Algorytmy-Geometryczne)

2. **Specyfikacji używanego sprzętu**
   a. **środowisko:** Visual Studio Code / Jupyter Notebook
   b. **systemem operacyjny:** Windows 10
   c. **procesor:** Ryzen 5 4600H architektury x86-64

3. **Moduły programu**

a. **Ręczne zadawanie punktów**

Część ta umożliwia ręczne zadawanie chmury punktów za pomocą funkcji:

```
def add_polygon(polygon_manual):
```

Przekazujemy jej tablicę *poligon_manual*, do której zapisane zostaną punkty w postaci krotki współrzędnych.

b. **Triangulacja Delaunay'a**

Część ta zawiera algorytm wizualizujący algorytm Delaunay'a:

```
def del_vis(del_triangulation, points):
```

del_triangulation to triangulacji Delaunay'a zwracana przez funkcję *Delaunay()* z biblioteki *scipy*; *points* to zbiór punktów tworzących wielokąt.

c. **Triangulacja Earcut**

Część ta zawiera algorytm wizualizujący algorytm Earcut:

```
def draw_earcut(verts, simplices):
```

verts to tablica tablic dwuelementowych zawierających współrzędne x, y punktów tworzących wielokąt; *simplices* to tablica tablic zawierająca indeksy punktów z tablicy *verts*.

d. **Outer space**

Część ta zawiera algorytm tworzący trójkąt zawierający chmurę punktów *points*:

```
def add_outer_space(points):
```

funkcja ta zwraca tablicę krotek punktów, stanowiących wierzchołki trójkąta.

e. **Funkcje pomocnicze**

Część ta zawiera funkcje pomocnicze dla algorytmu kirkpatricka'a:

```
def get_independent_set(polygon, triangles, outer_space):
```

Funkcja ta przyjmuje:

- tablicę punktów *polygon*;
- tablicę trójkątów stanowiących obecną triangulację *triangles*;
- trójkąt zewnętrzny *outer_space*.

Funkcja zwraca:

- tablicę krotek punktów, stanowiących zbiór punktów niezależnych.

```
def remove_independent_set(polygon, triangles, independent_set, outer_space):
```

Funkcja ta przyjmuje:

- zbiór punktów *polygon*;
- tablicę trójkątów stanowiących obecną triangulację *triangles*;
- zbiór punktów stanowiących zbiór niezależny *independent_set*;
- trójkąt zewnętrzny *outer_space*.

Funkcja zwraca:

- tablicę tablic krotek punktów, stanowiących zbiory punktów, będące sąsiadami usuniętych punktów;
- tablicę usuniętych trójkątów;
- zbiór punktów pomniejszoną o punkty usunięte.

```
def triangle_intersection(tri1, tri2):
```

Funkcja ta przyjmuje:

- dwie krotki punktów stanowiących trójkąty *tri1* i *tri2*.

Funkcja zwraca:

- True jeśli część wspólna trójkątów posiada niezerowe pole, a False w przeciwnym wypadku

f. **Kirkpatrick**

Część ta zawiera algorytm Kirkpatrick'a:

```
def kirkpatrick(polygon):
```

Funkcja ta przyjmuje:

- tablicę punktów *polygon*;

Funkcja zwraca:

- słownik definiujący relacje między trójkątami stanowiącymi kolejne (mniej złożone) triangulacje
- trójkąt stanowiący część słownika, od którego funkcja locate rozpocznie przeszukiwanie słownika
- triangulację delaunay'a

g. **Locate**

Część ta zawiera algorytm Kirkpatrick'a:

```
def locate(triangles_graph, outer_space,
point):
```

Funkcja ta przyjmuje:

- słownik definiujący relacje między trójkątami stanowiącymi kolejne (mniej złożone) triangulacje zwracany przez funkcję kirkpatrick *triangles_graph*;
- trójkąt zewnętrzny *outer_space*.
- krotkę ze współrzędnymi punktu, którego pozycję chcemy określić *point*.

Funkcja zwraca:

- trójkąt z bazowej triangulacji w którym znajduje się punkt point

h. **Wizualizacja**

Część ta zawiera funkcje:

```
def kirkpatrick_vis(polygon)

def locate_vis(triangles_graph, outer_space, point, all_triangulations)

def locate_vis2(triangles_graph, outer_space, point, all_triangulations)
```

Są to te same funkcje co opisane w punktach f, g jednak wizualizują one kroki algorytmu. Funkcja *kirkpatrick_vis* zwraca dodatkowo tablicę zbiorów odcinków kolejnych triangulacji uzyskanych z algorytmu kirkpatrik'a. Do poprawnego działania funkcji *locate_vis* i *locate_vis2*, przekazujemy im dodatkowy parametr opisany powyżej (konieczne jest wywołanie najpierw *kirkpatrick_vis*)

i. **Działanie**

Część ta zawiera wywołanie fukcji *kirkpatrick* i *locate* dla wybranego przypadku

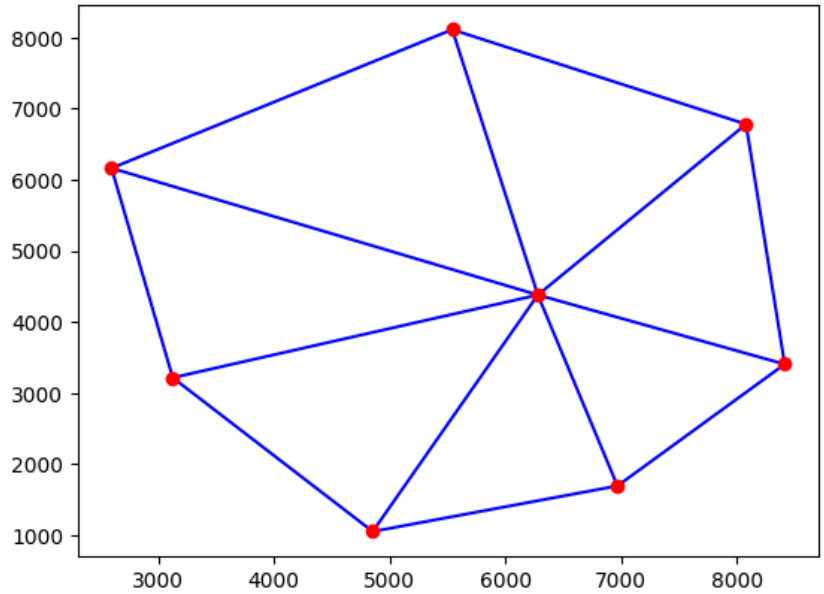
4. Działanie programu

a. **Ręczne zadawanie wierzchołków**

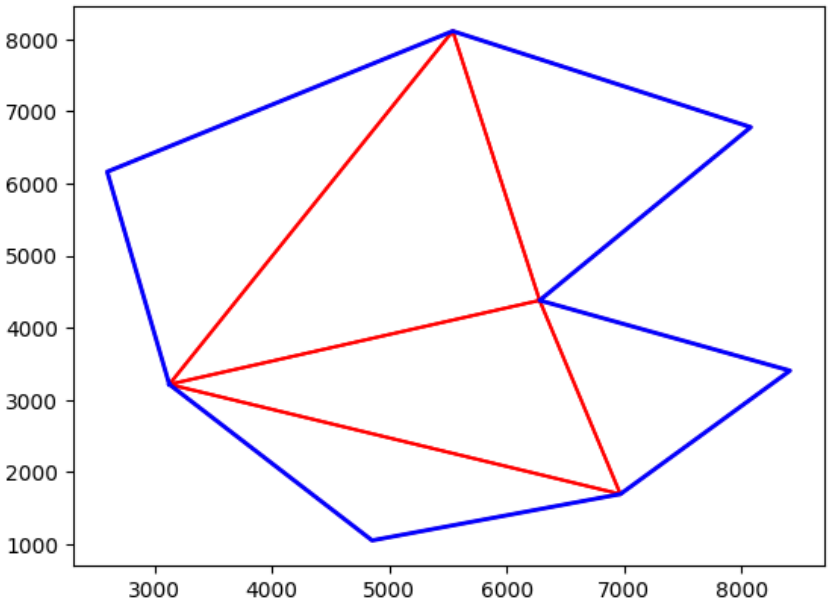
Wywołanie funkcji add_polygon umożliwia zadanie chmury wierzchołków do przetestowania algorytmu kirkpatrika i lokalizacji punktu

b. **Triangulacja Delaunay'a i Earcut**

Część ta przedstawia efekty algorytmów triangulacji Delaunay'a i Earcut. Do wyznaczenia triangulacji wykorzystano funkcje biblioteczne. Przykładowy efekt algorytmu Delaunay'a przedstawiono na wykresie 1, a algorytmu Earcut na wykresie 2.



Wykres 1 Efekt algorytmu Delaunay'a



Wykres 2 Efekt algorytmu Earcut

c. **Implementacja algorytmu kirkpatricka, locate i funkcji pomocniczych**

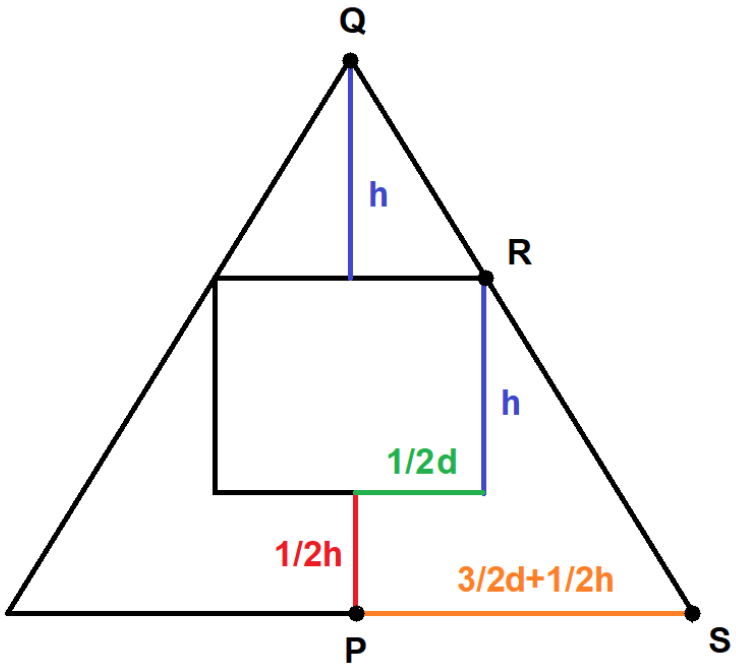
i. **Wyznaczenie zewnętrznego trójkąta**

W odniesieniu do rysunku 1 przedstawiającego relacje między zbiorem punktów znajdującym się w prostokącie, a trójkątem zewnętrznym, przyjmując współrzędne (0,0) dla punktu P otrzymujemy współrzędne punktów Q, R, S:

Q=(0, 5/2h)

R=(1/2d, 3/2h)

S=(3/2d+1/2h,0)



Rysunek 1.

Wszystkie punkty znajdują się w prostokącie, więc ponieważ $h > 0$ to punkty nie znajdą się poniżej dolnego boku trójkąta. Aby wykazać, że punkty nie znajdą się za bokiem QS, znajdźmy równanie prostej na której on leży. Ma ono postać:

$$y = -\frac{\frac{5}{2}h}{\frac{3}{2}d+\frac{1}{2}h}x + \frac{5}{2}h.$$

Chcemy aby punkt R znajdował się poniżej prostej, więc otrzymujemy nierówność:

$$y \leq -\frac{\frac{5}{2}h}{\frac{3}{2}d+\frac{1}{2}h}x + \frac{5}{2}h.$$

Podstawiając do niej punkt R otrzymujemy:

$$\frac{3}{2}h \leq -\frac{\frac{5}{2}h}{\frac{3}{2}d+\frac{1}{2}h}\frac{1}{2}d + \frac{5}{2}h.$$

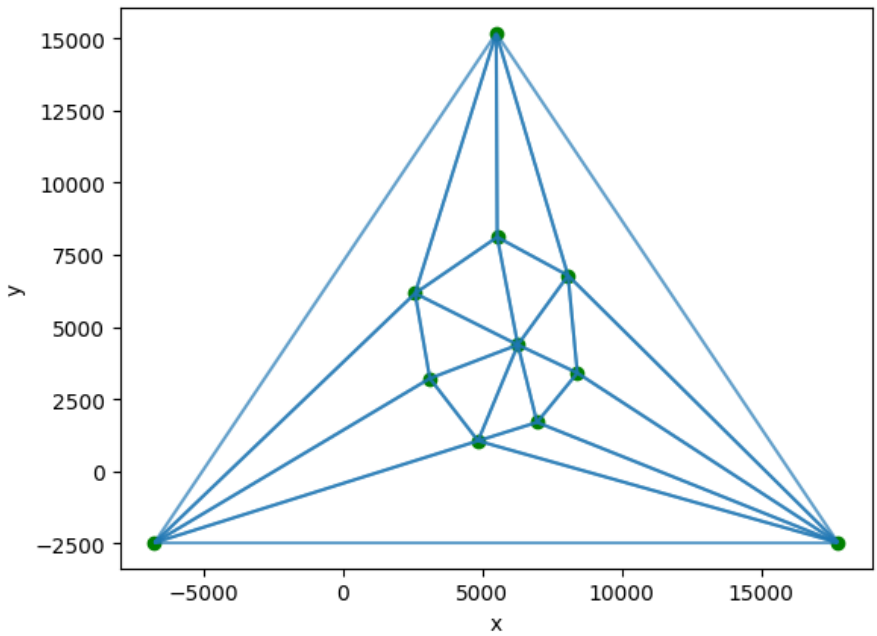
Przekształcając je możemy dojść do nierówności:

$$0 \leq \frac{1}{4}hd + \frac{5}{2}h^2.$$

Jest ono zawsze spełnione dla $h, d > 0$, więc punkty nie mogą wyjść poza bok QS, a dzięki symetryczności pozostałego boku trójkąta i QS względem PQ, za pozostały również. Wyznaczanie trójkąta zewnętrznego jest więc poprawne.

ii. Podstawowa triangulacja

Do podstawowej triangulacji używany jest algorytm Delaunay'a. Przykładową triangulację zaprezentowano na wykresie 3.

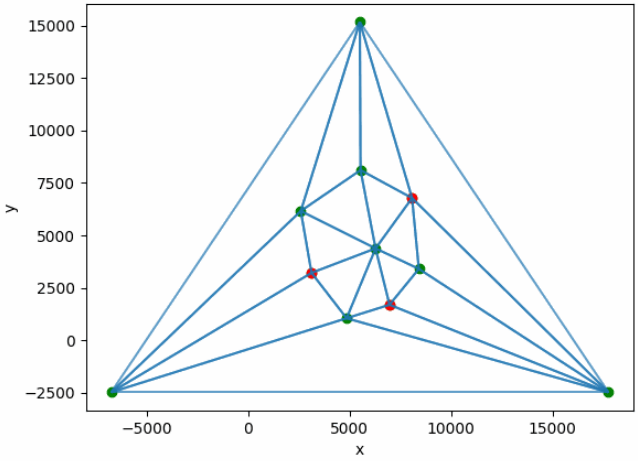


Wykres 3 Przykładowa triangulacja

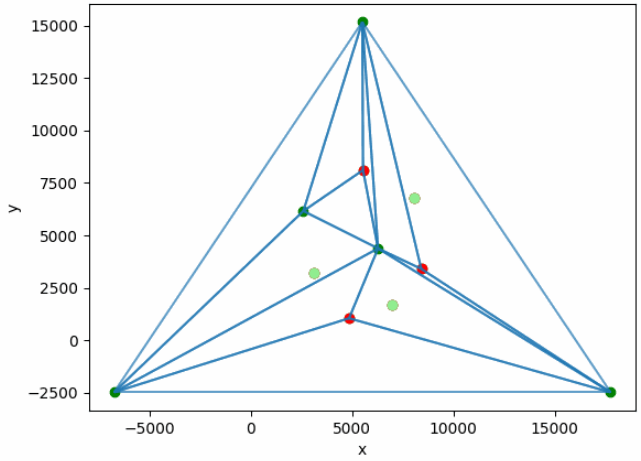
iii. Tworzenie sekwencji triangulacji

1. Wyznaczenie zbioru niezależnego

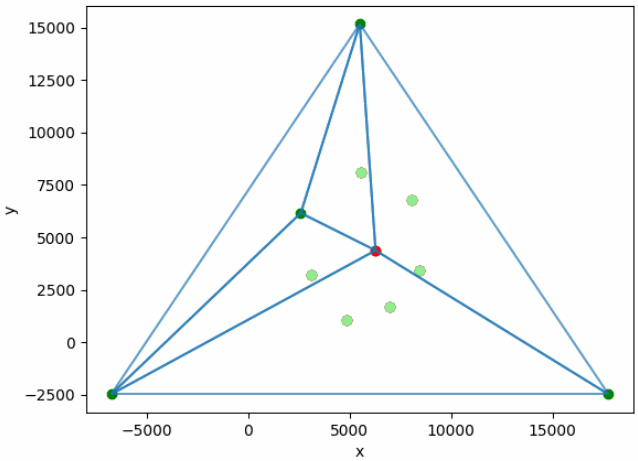
Zbiór niezależny wyznaczany jest spośród punktów niebędących wierzchołkami trójkąta zewnętrznego. Na początku wybierany jest dowolny punkt, a następnie dla każdego pozostałego punktu, jeśli nie sąsiaduje z już wybranymi punktami, wybieramy go także. Operację powtarzamy do sprawdzenia wszystkich punktów. Przykładowe kolejne zbiory niezależna zaznaczono na wykresach 4.1-4.4



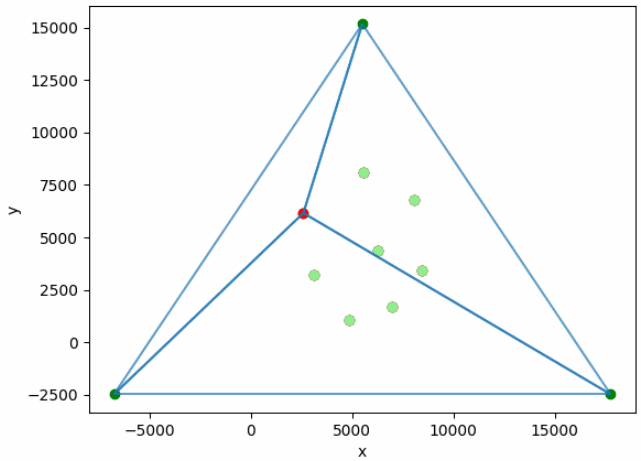
Wykres 4.1



Wykres 4.2



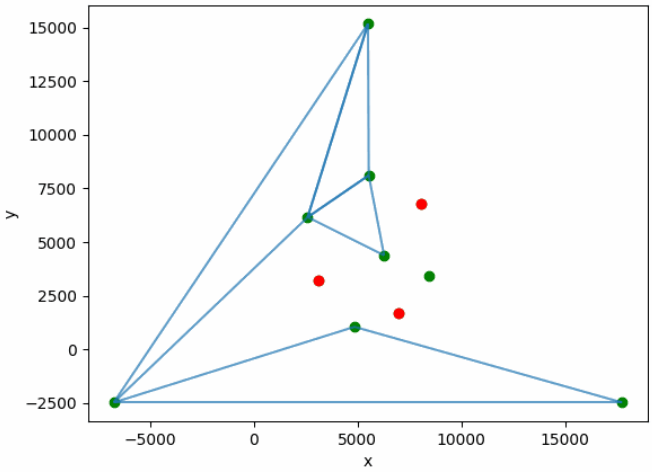
Wykres 4.3



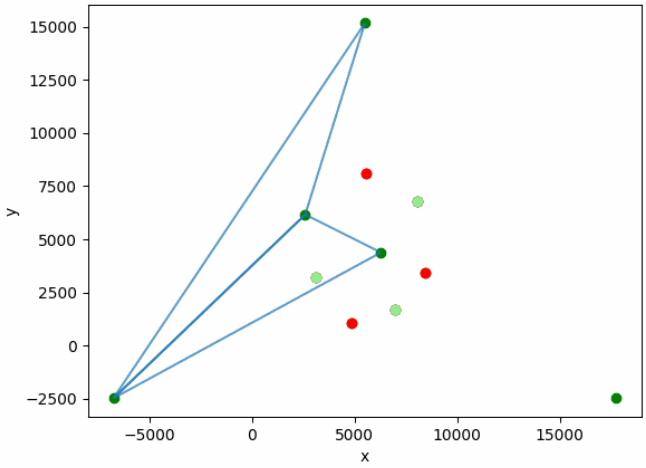
Wykres 4.4

2. Usunięcie zbioru niezależnego

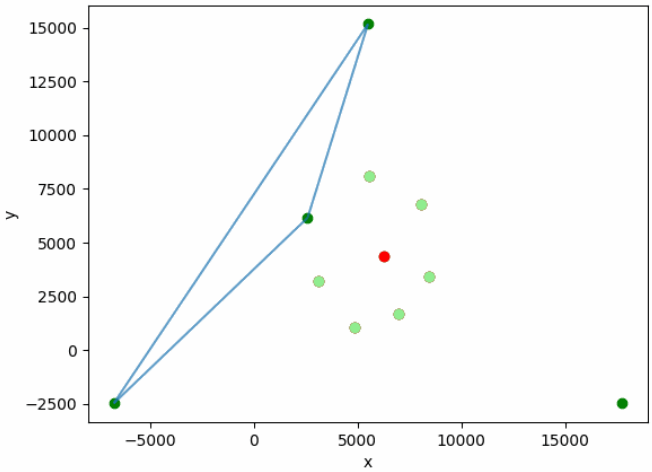
Spośród zbioru punktów usuwamy wybrane punkty oraz trójkąty, których te punkty były wierzchołkami. Przykładowe kolejne usunięte zbiory punktów niezależnych i trójkątów zaprezentowano na wykresach 5.1-5.4



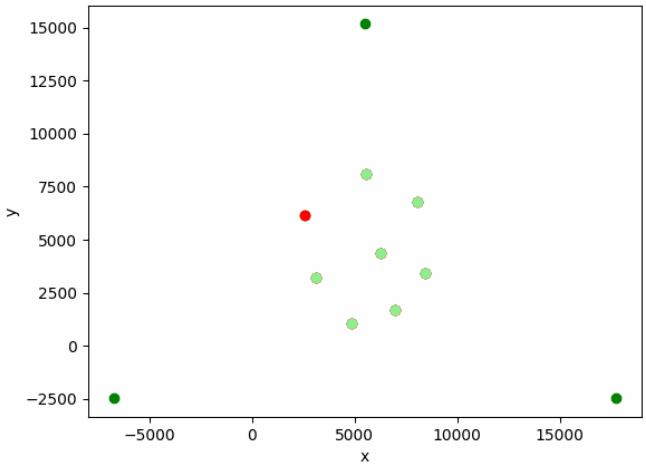
Wykres 5.1



Wykres 5.2



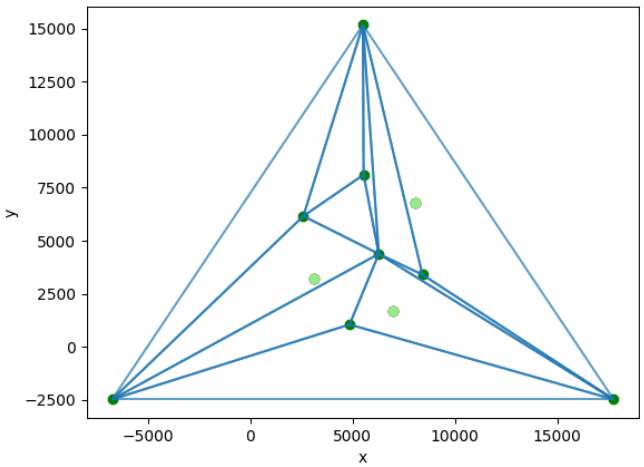
Wykres 5.3



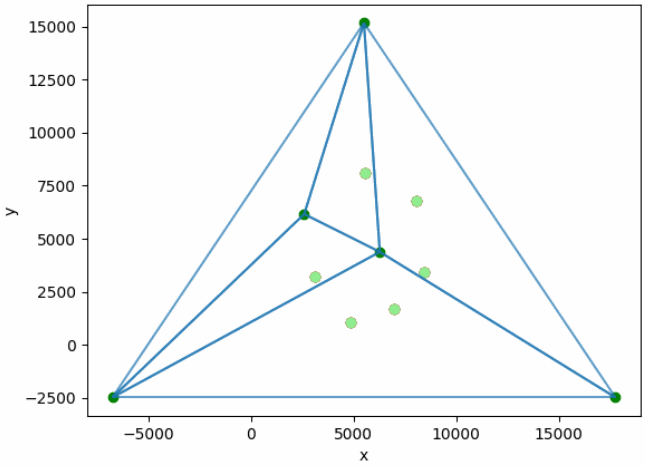
Wykres 5.4

3. **Tworzenie nowej triangulacji**

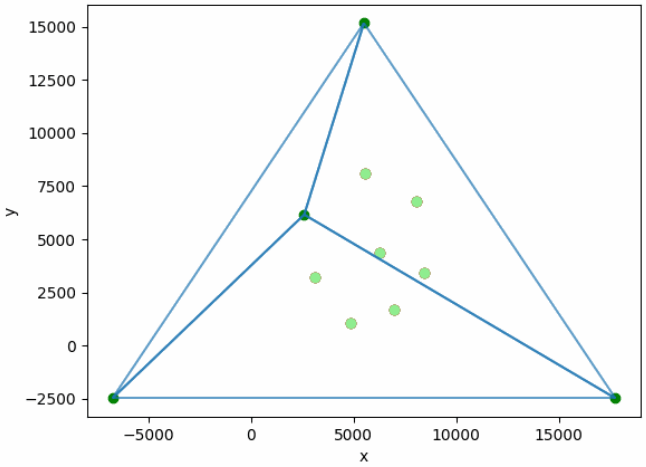
   Dla sąsiadów każdego usuwanego punktu tworzymy triangulacje algorytmem Earcut, co zapewnia utworzenie nowej, pełnej triangulacji pozostałych punktów. Przykładowe kolejne triangulacje zaprezentowano na wykresach 6.1-6.4



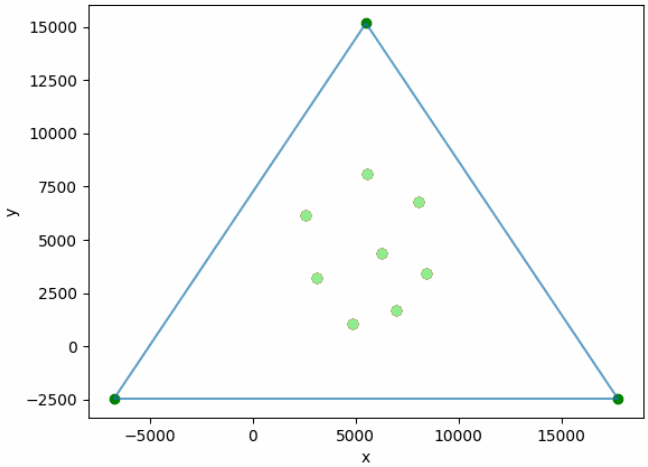
Wykres 6.1



Wykres 6.2



Wykres 6.3



Wykres 6.4

d. **Implementacja algorytmu kirkpatricka i locate z wizualizacją**
e. **Wywołanie algorytmów**
f.

5. **Korzystanie z programu**

Program umożliwia zadawanie zbioru punktów w module *Ręczne zadawanie punktów*. Po uruchomieniu się okna z wykresem, możemy zadawać punkty, klikając lewym przyciskiem. Aby zapisać punkty (w parametrze

przekazywanym do funkcji) konieczne jest użycie klawisza „Escape”!

Aby wykonać algorytm kirkpatrick’a i locate oraz ich wersji z wizualizacją dla zadanego zbioru punktów, konieczne jest, aby w module *Działanie* zmiennej „chosen_polygon” przypisano wartość zmiennej „poligon_manual”

Funkcje locate_vis i locate_vis2 wymagają wcześniejszego uruchomienia kirkpatrick_vis, aby uzyskać tablicę kolejnych triangulacji.

6. **Sprawozdanie**
   a. Opis problemu

      - Problemem, który chcemy rozwiązać, jest lokalizacja punktu w przestrzeni dwuwymiarowej z podziałem poligonowym, będącym dokładnie podziałem płaszczyzny na trójkąty.

      - Parametrem wejściowym jest zbiór punktów rzędu n.

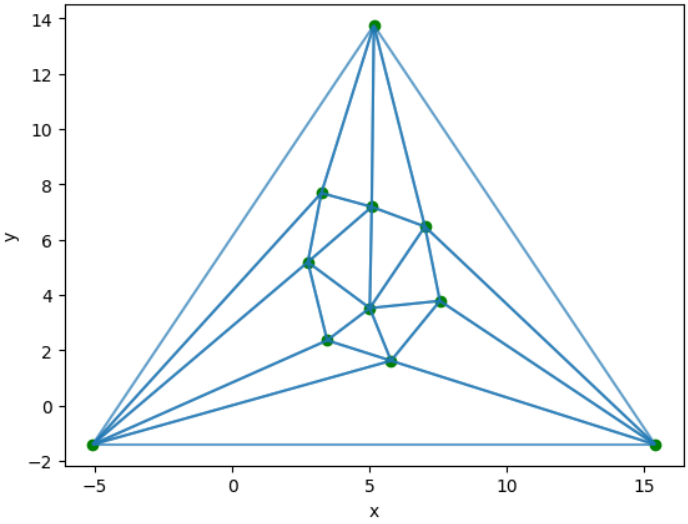
      - Problem ten ma trywialne rozwiązanie w czasie O(n) polegającym na sprawdzeniu po kolej wszystkich trójkątów i sprawdzenie, czy punkt się w nim nie znajduje.

      - Zastosowanie algorytmu kirkpatrk’a wymaga preprocessingu o złożoności O(n log n), jednak po jego jednokrotnym wykonaniu, możemy uzyskać lokalizację punktu w czasie O(log n)

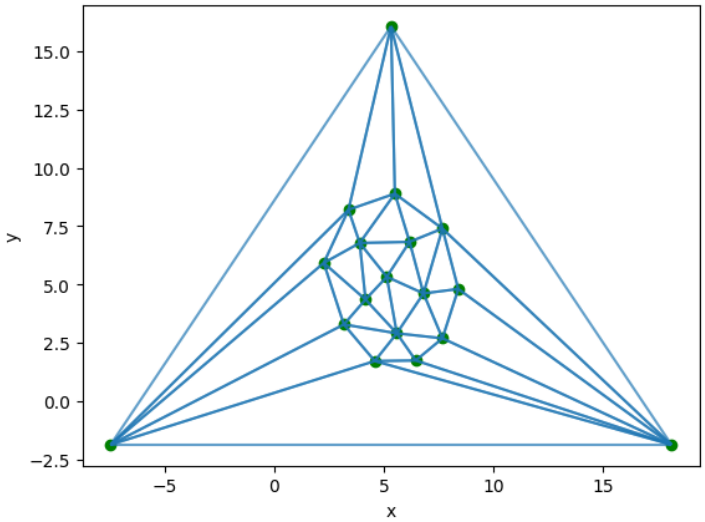
   b. Testy i ich wyniki

      Przeprowadzone testy w punktach i. oraz ii. dla czterech chmur punktów. Na wykresach 7.1, 7.2, 8.1, 8.2 współrzędne punktów były liczbami zmiennoprzecinkowymi. Na wykresach 7.3, 7.4, 8.3, 8.4 były to liczby całkowite.

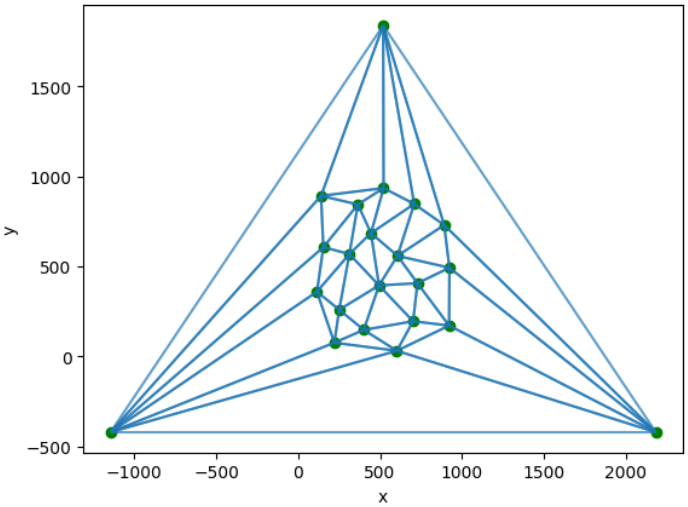
i. Wyniki testu poprawności triangulacji dla kilku chmur punktów przedstawiono na wykresach 7.1-7.4.



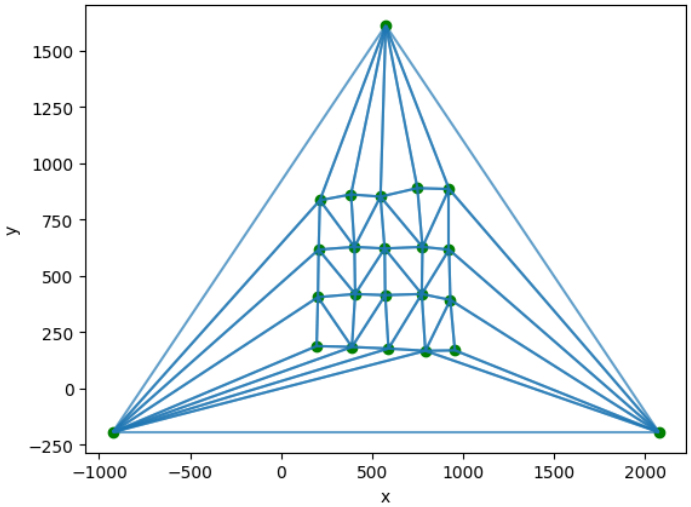
Wykres 7.1



Wykres 7.2

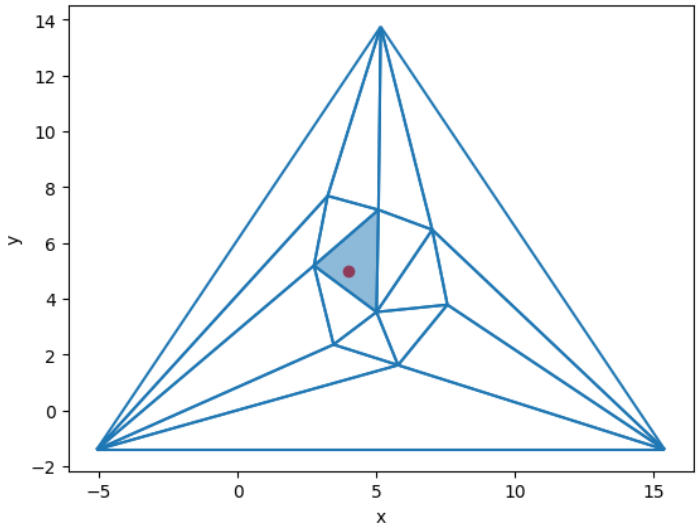


Wykres 7.3

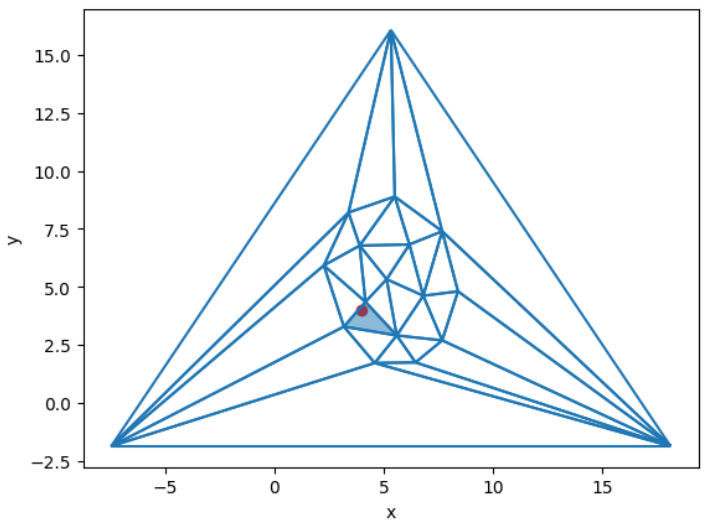


Wykres 7.4

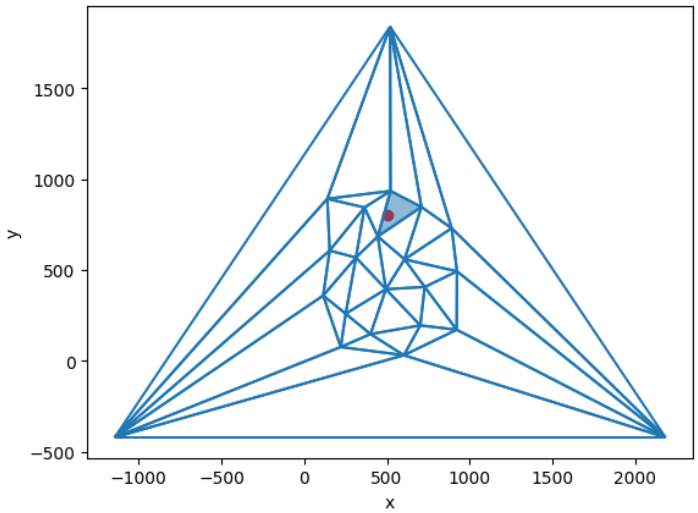
ii. Wyniki testu poprawności lokalizacji punktu dla kilku chmur punktów przedstawiono na wykresach 8.1-8.4.



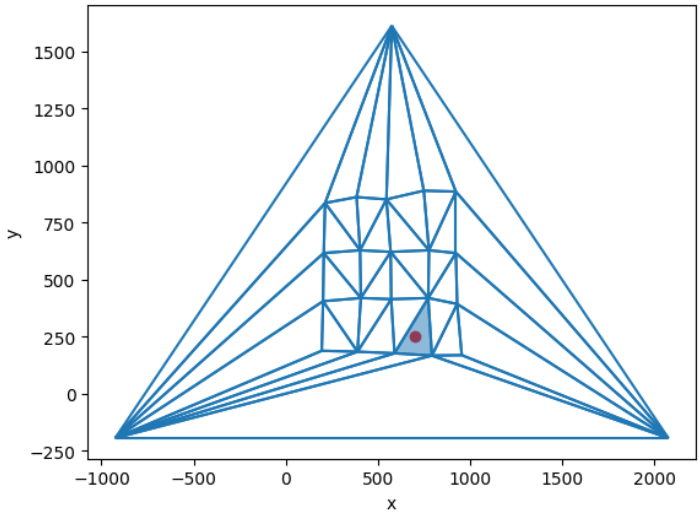
Wykres 8.1



Wykres 8.2

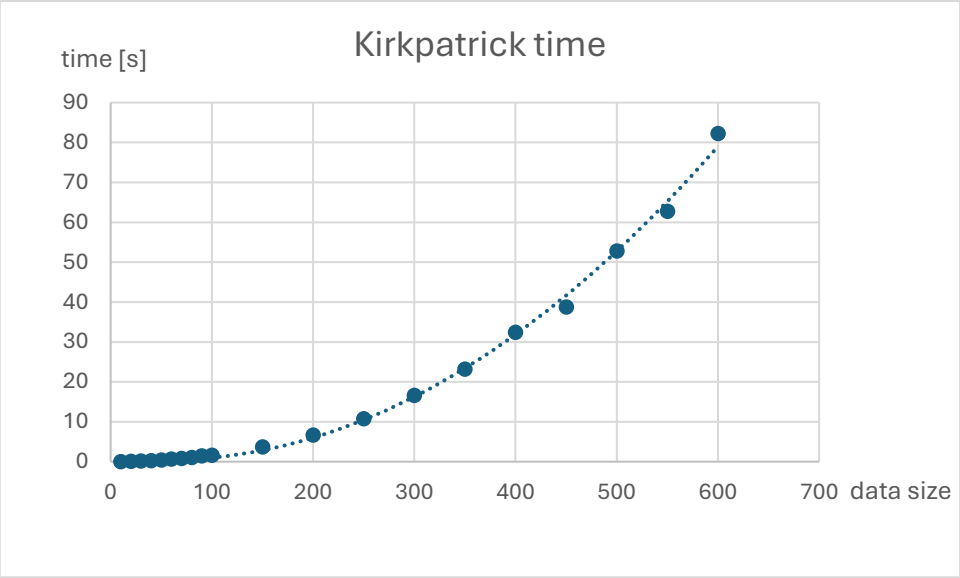


Wykres 8.3

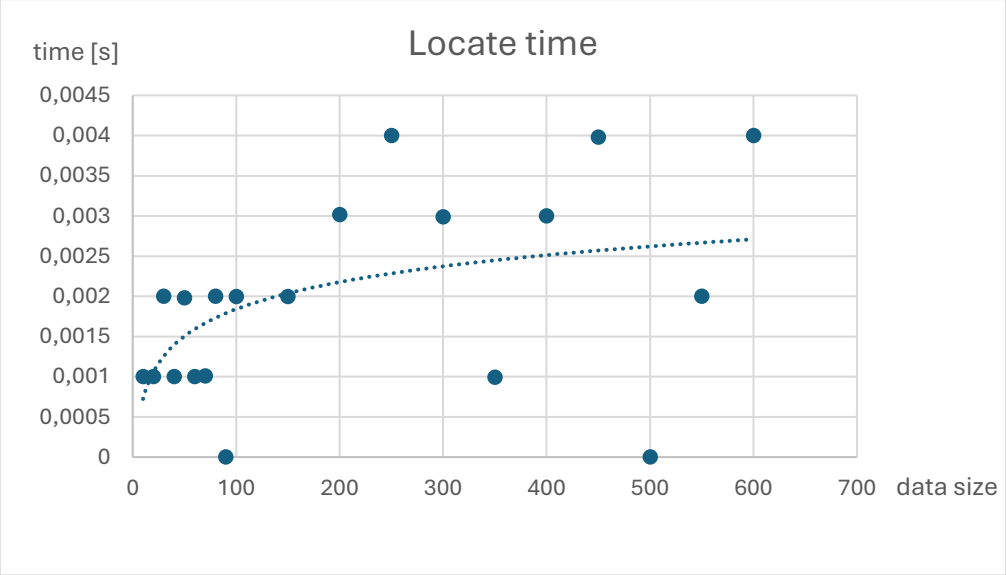


Wykres 8.4

iii. Wyniki czasów wykonywania się algorytmu, wraz z linią trendu algorytmu kirkpatrick'a pokazano na wykresie 9.1, a algorytmu locate na wykresie 9.2. Ponieważ czas wykonywania się algorytmu locate jest bardzo krótki, bliski dokładności funkcji time, wyniki są mało dokładne, jednak mogą wykazywać tendencje do złożoności O(log n)



Wykres 9.1 Czas wykonywania algorytmu Kirkpatricka



Wykres 9.2 Czas wykonywania algorytmu locate

7. Źródła

https://ics.uci.edu/~goodrich/teach/geom/notes/Kirkpatrick.pdf

Slajdy z wykładu